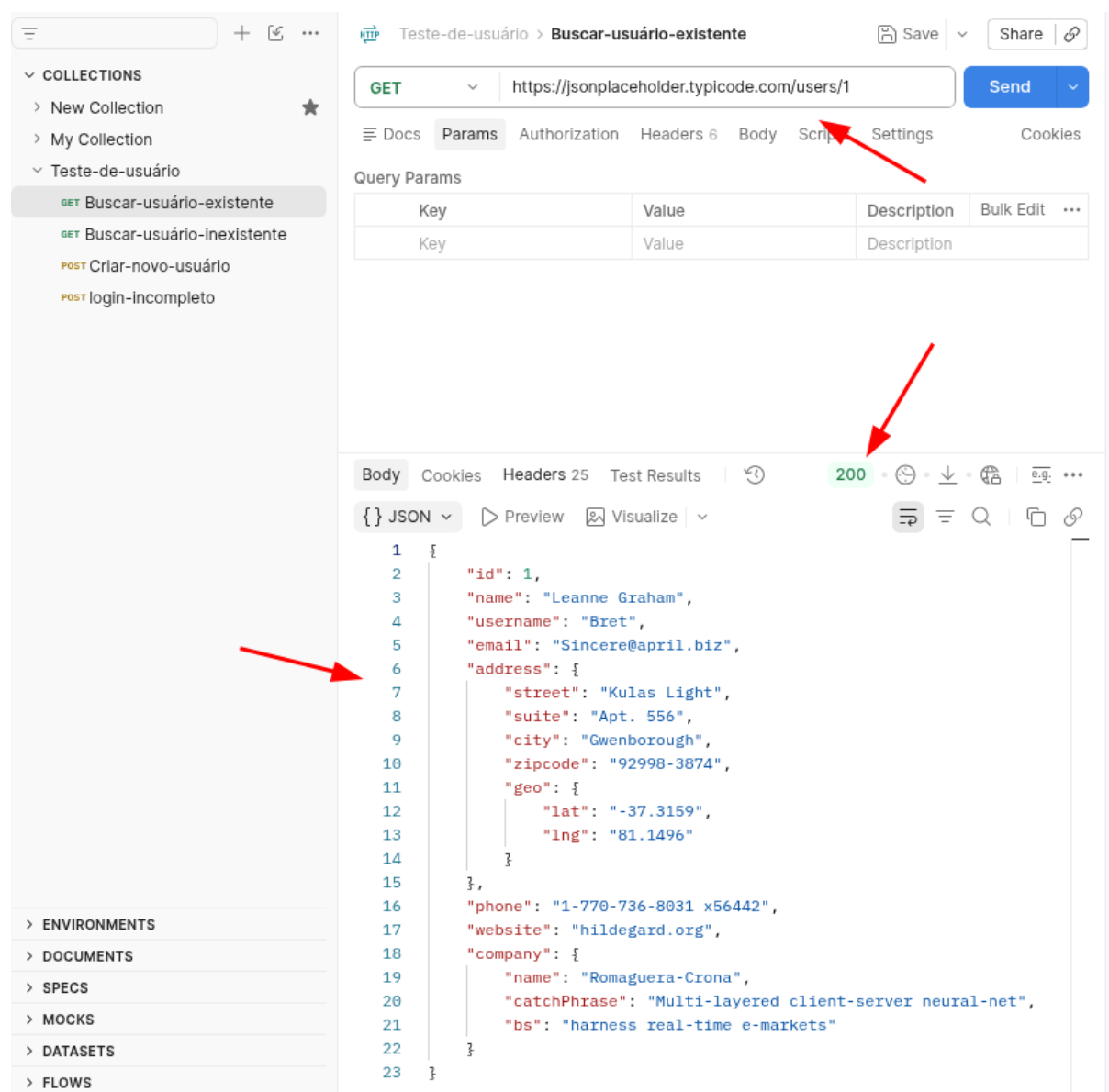


Testes de Api, consultas ao banco de dados e testes de automação

Objetivo: Validar o comportamento de uma API pública (alternativa) através de testes com os métodos **GET** e **POST**, cobrindo cenários de sucesso e de erro, utilizando o Postman como ferramenta de execução.

Grupo 1 – Testes GET

O método **GET** foi utilizado para validar a consulta de dados na API.



The screenshot displays the Postman interface for a GET request. The left sidebar shows a collection named 'Teste-de-usuário' with several items, including 'Buscar-usuário-existente' which is selected. The main area shows the request details for 'Buscar-usuário-existente' with the URL 'https://jsonplaceholder.typicode.com/users/1'. The 'Query Params' table is empty. The 'Body' tab is active, showing a JSON response. A red arrow points to the 'Script' tab, and another red arrow points to the '200' status code. A third red arrow points to the 'JSON' tab in the response viewer.

Teste-de-usuário > Buscar-usuário-existente

GET https://jsonplaceholder.typicode.com/users/1

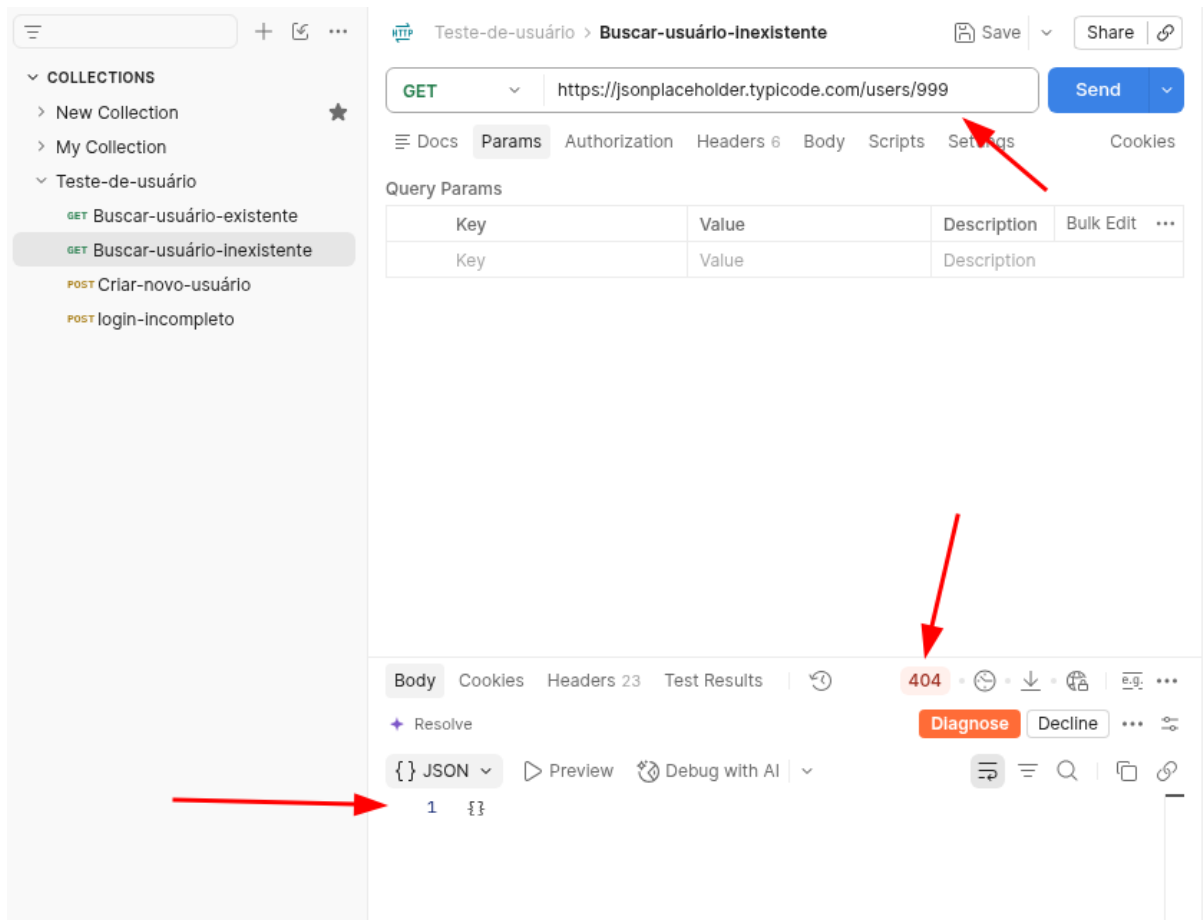
Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 25 Test Results 200

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
```

Consulta de usuário existente, 200 Ok, requisição bem-sucedida, dados retornados corretamente

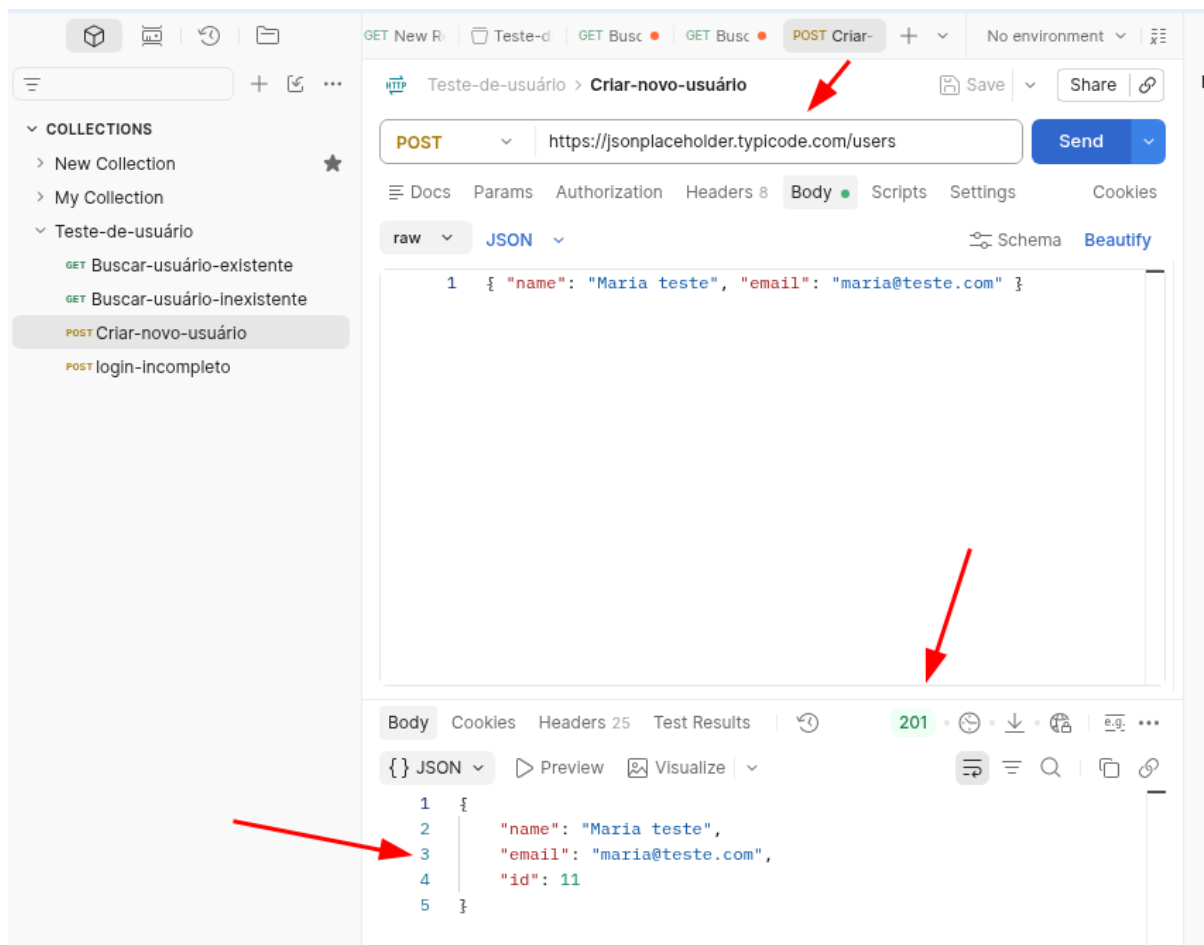


consulta de usuário inexistente (ID 999) retorno 404, retorno vazio, confirmando que o recurso não existe na base

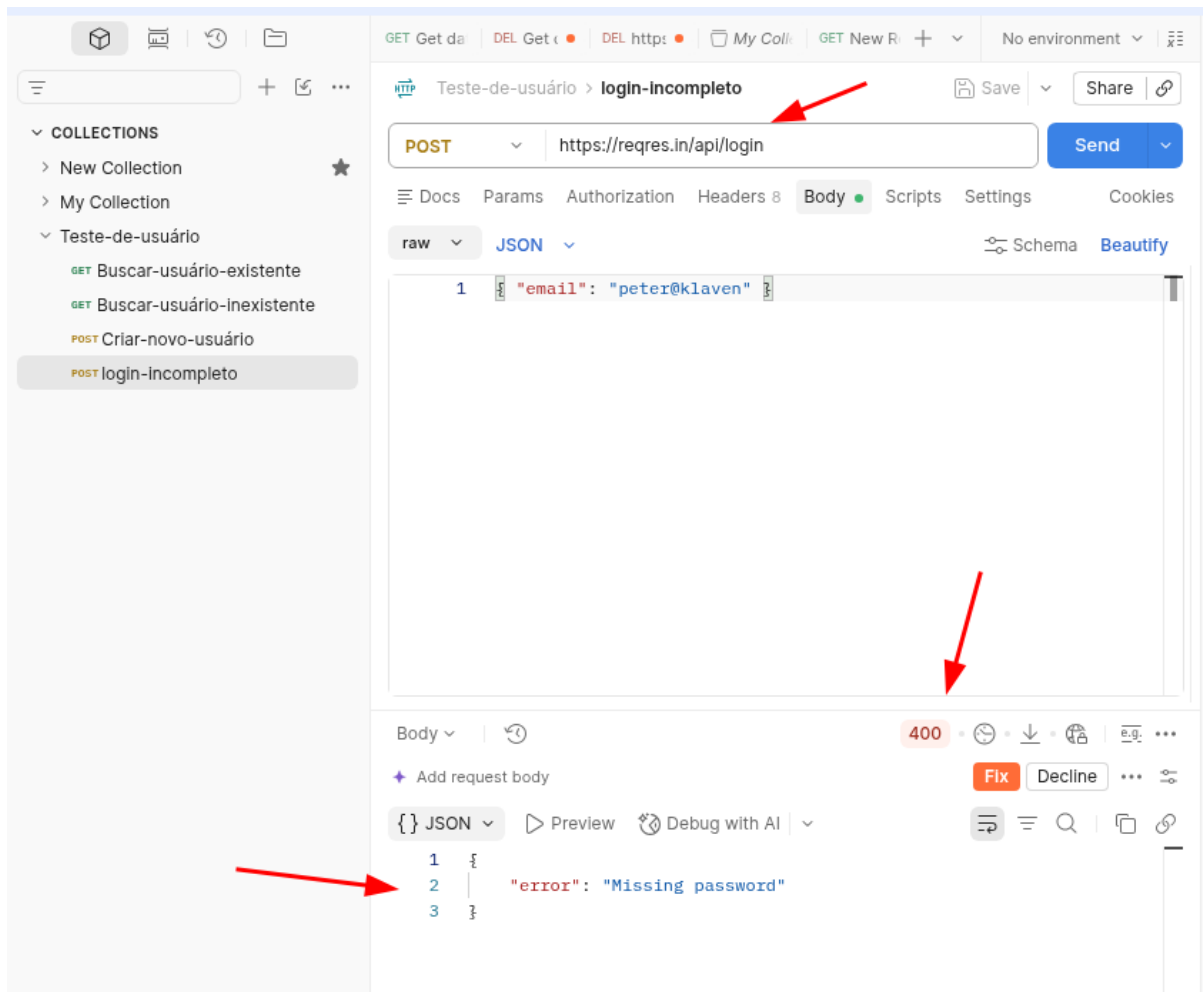
Análise: Os testes confirmam que a API responde corretamente tanto para dados válidos quanto para registros inexistentes, validando o tratamento adequado de recursos não encontrados.

Grupo 2 – Testes POST

O método **POST** foi utilizado para validar a criação de novos registros.



Criação de registro com todos os campos obrigatórios, 201 informando criação bem-sucedida.



Criação de registro sem o campo "senha" (apenas e-mail informado), erro 400, API rejeita a requisição por campo obrigatório ausente.

Análise: Os testes validam que a API não apenas cria registros corretamente, como também aplica as validações de campos obrigatórios, retornando erro apropriado quando dados essenciais estão ausentes.

Conclusão

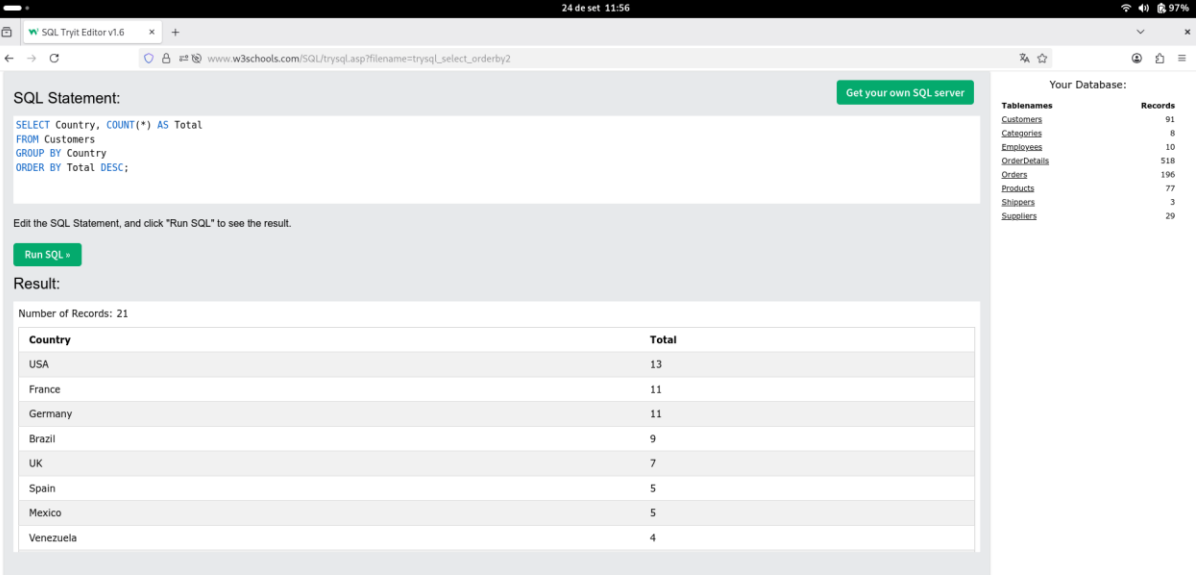
Os testes realizados demonstram a importância de validar não somente os cenários de sucesso (*happy path*), mas também os cenários de erro, garantindo que a API trate corretamente requisições inválidas ou incompletas. Esse tipo de teste é essencial para o QA, pois permite identificar falhas de validação e comportamento inesperado antes que cheguem à produção, evitando bugs que passariam despercebidos em testes apenas manuais/visuais.

Consulta ao Banco de dados

Foram realizadas consultas em um banco de dados online próprio, com o objetivo de validar a integridade das informações persistidas — sem duplicidade de registros e sem dados incorretos.

As consultas incluíram filtro por grupo específico (ex: clientes do Brasil), ordenação alfabética e contagem de registros, tanto total quanto separada por grupo.

Evidências:



The screenshot shows the SQL Tryit Editor interface. The SQL Statement is:

```
SELECT Country, COUNT(*) AS Total
FROM Customers
GROUP BY Country
ORDER BY Total DESC;
```

The Result shows 21 records. The table displays the following data:

Country	Total
USA	13
France	11
Germany	11
Brazil	9
UK	7
Spain	5
Mexico	5
Venezuela	4

On the right side, the 'Your Database' section lists the following tables and their record counts:

Tablenames	Records
Customers	91
Categories	8
Employees	10
OrderDetails	518
Orders	196
Products	77
Suppliers	3
Suppliers	29



The screenshot shows the SQL TryIt Editor v1.6 interface. The SQL Statement is: `SELECT * FROM Customers WHERE Country = 'Brazil';`. The result shows 9 records. The interface includes a 'Run SQL' button and a 'Your Database' sidebar listing tables and their record counts.

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
15	Comércio Mineiro	Pedro Afonso	Av. dos Lusíadas, 23	São Paulo	05432-043	Brazil
21	Família Arquibaldo	Aria Cruz	Rua Orós, 92	São Paulo	05442-030	Brazil
31	Gourmet Lanchonetes	André Fonseca	Av. Brasil, 442	Campinas	04876-786	Brazil
34	Hanari Carnes	Mario Pontes	Rua do Paço, 67	Rio de Janeiro	05454-876	Brazil
61	Que Delícia	Bernardo Batista	Rua da Panificadora, 12	Rio de Janeiro	02389-673	Brazil
62	Queen Cozinha	Lúcia Carvalho	Alameda dos Canários, 891	São Paulo	05487-020	Brazil
67	Ricardo Adocicados	Janete Limeira	Av. Copacabana, 267	Rio de Janeiro	02389-890	Brazil
81	Tradição Hipermercados	Anabela Domingues	Av. Inês de Castro, 414	São Paulo	05634-030	Brazil

Análise: A consulta direta ao banco permite validar a camada de persistência, indo além do retorno da API, identificando inconsistências que passariam despercebidas apenas pela interface.

Testes Automatizados

Foram realizados 2 testes automatizados end to end: **Maestro** (mobile) e **Cypress** (web).

No **Maestro**, o fluxo cobre abrir o app, tocar no menu de login, preencher usuário e senha e validar que o acesso foi concluído.

No **Cypress**, o fluxo cobre acessar a página, preencher usuário e senha válidos, clicar em entrar e validar a exibição da mensagem de sucesso.

Conclusão

A automação permite executar testes repetitivos com rapidez e consistência, liberando o QA para focar em cenários exploratórios e de maior complexidade. Combinada aos testes manuais de API e banco de dados, ela fortalece a cobertura geral, reduzindo o risco de bugs chegarem à produção.

Evidencia:

Cypress: https://drive.google.com/file/d/1x_uO_a0wjw_bO3lM6MlPL-d1axj2gCk-/view?usp=drive_link

Maestro:

https://drive.google.com/file/d/1HyaO6Z03w7Ph90NgWUx91PmiXTFQ2EVh/view?usp=drive_link

Ambiente de teste:

Banco de dados:

https://www.w3schools.com/sql/trysql.asp?filename=trysql_select_all

Cypress: <https://the-internet.herokuapp.com/login>

Autor: Wesley santos